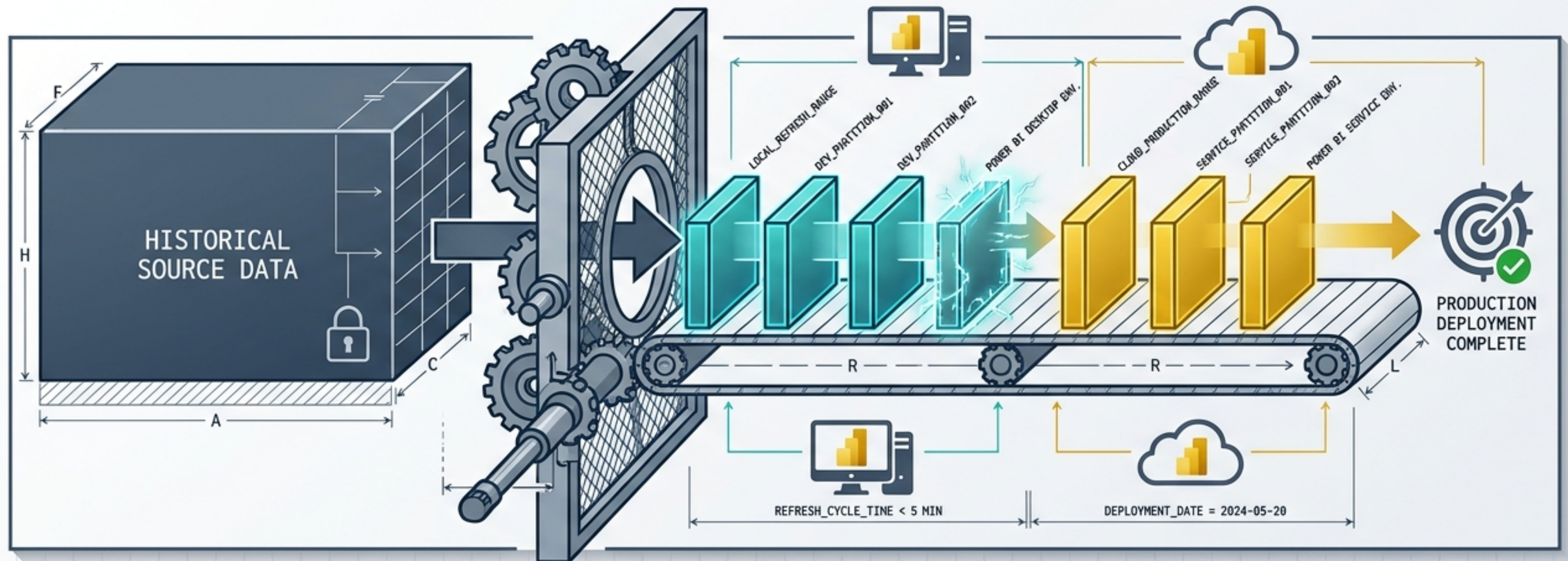


Mastering Incremental Refresh

A visual diagnostic guide for the **PL-300 exam** and production deployment



Incremental refresh solves the performance bottlenecks of massive datasets



Faster Refreshes

Gets **high speed** data optimized for telemetry dashboard.

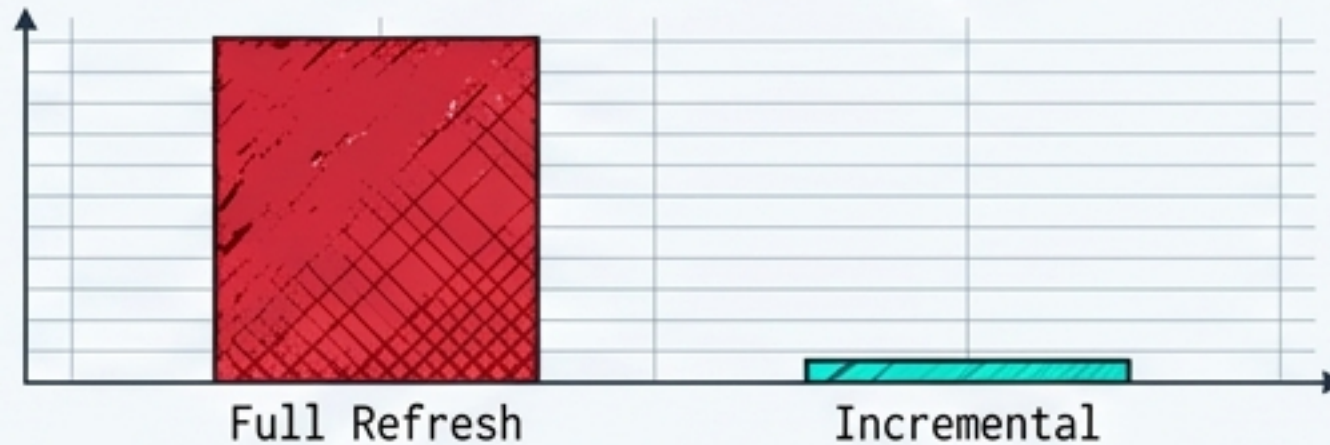
Only **new or modified** data is queried and **processed**, drastically reducing wait times.

Reliability



Shorter refresh cycles eliminate **network timeouts** and source system **locks**.

Resource Consumption



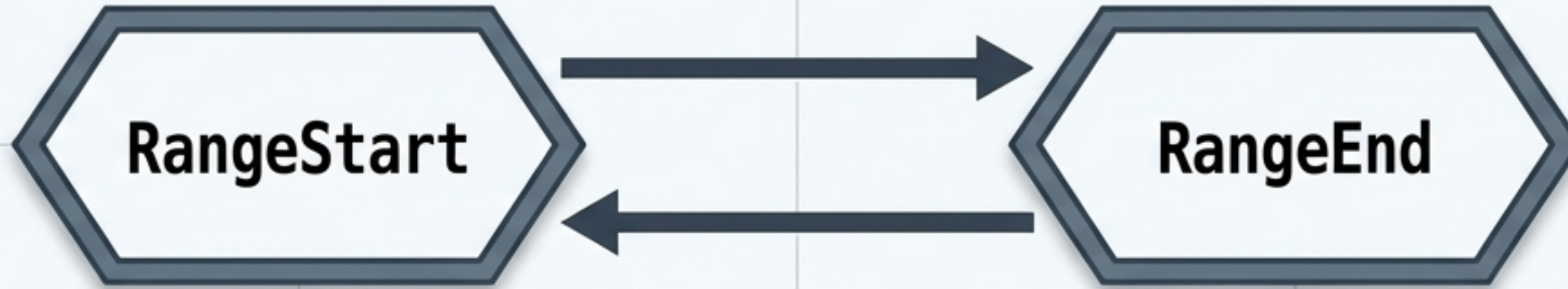
Minimizes **CPU**, **memory**, and **network tax** on both **Power BI capacity** and **data sources**.

Large Dataset Support



Bypasses standard **import size limits** to support **multi-year historical datasets**.

The engine is powered by two strictly governed Power Query parameters

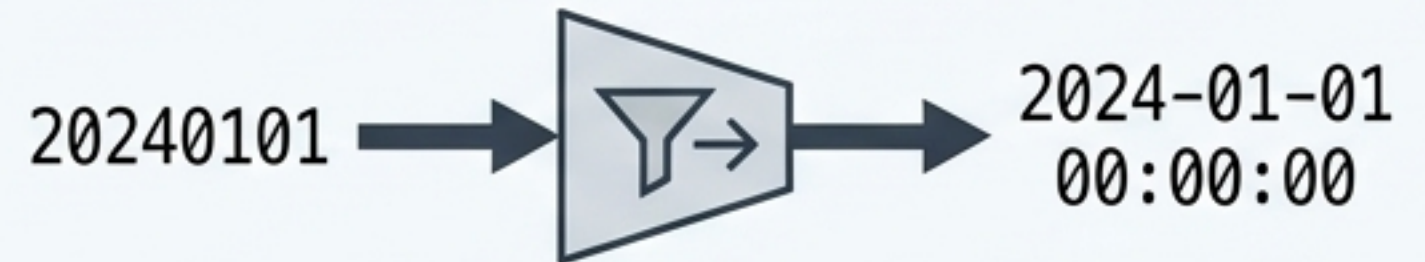


⚠ Exact Case-Sensitive Naming

~~rangestart~~ **RangeStart** ✓
~~Range_start~~ **RangeEnd** ✓

⚠ Strict Data Type Rule

Type:



Source columns stored as **Dates** or **Integers** must be converted to **DateTime** in Power Query while maintaining **Query Folding**.

Parameter behavior — fundamentally changes between local development and the cloud

Power BI Desktop (Development Phase)



- **Function:** Acts as a static, manual developer placeholder.
- **Example:** Hardcoded to 1 recent month (e.g., 2024-01-01 to 2024-02-01).
- **Result:** Keeps the local .pbix file small, lightweight, and fast during report design.

Power BI Service (Production Phase)



- **Function:** Dynamically overwritten by the automated partition engine.
- **Action:** Discards manual parameter dates upon publishing.
- **Result:** Generates automated, date-based partitions according to the configured refresh policy, querying only the active window and leaving history untouched.

Strict operator logic prevents record duplication at the midnight boundary

`[Date Column] >= RangeStart AND [Date Column] < RangeEnd`

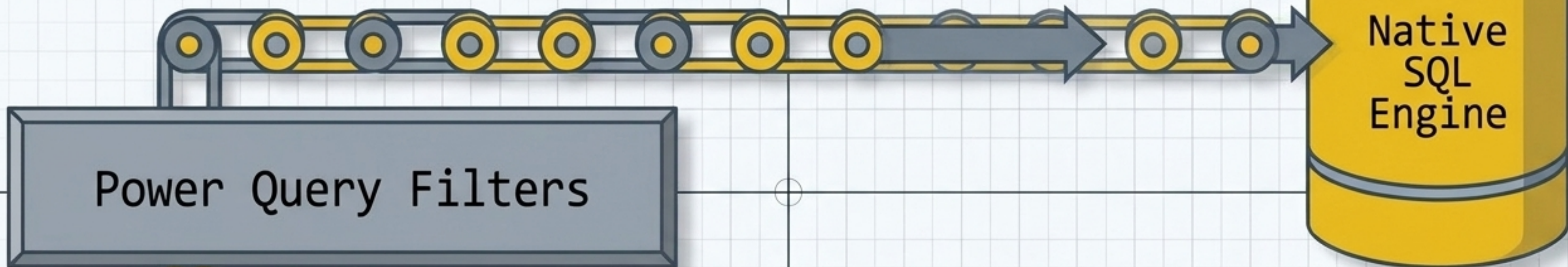


Why? Using `<=` on both ends would cause a single midnight record to duplicate across adjacent time partitions. The `>=` and `<` combination guarantees zero duplicate rows and zero missing records across boundaries.

Query folding must survive the parameter filter step

SUCCESS (FOLDING PRESERVED)

Essential for source-side query filtering during refreshes.



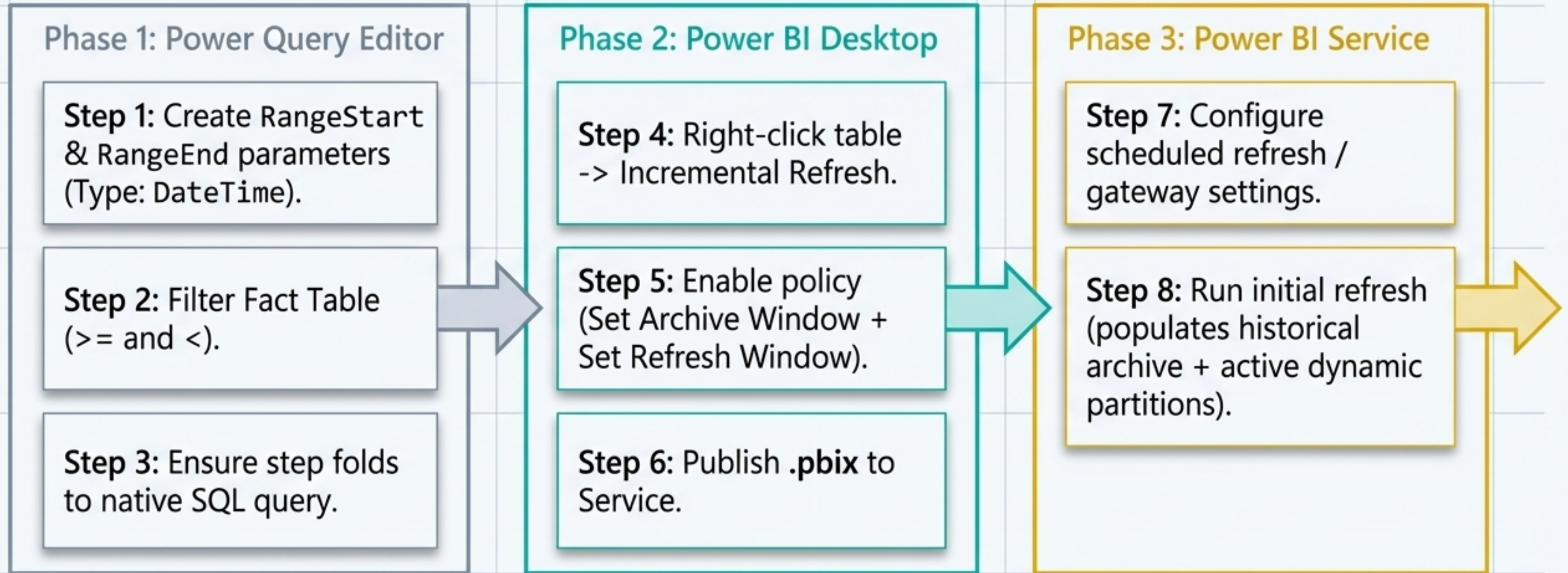
Power Query Filters

Native SQL Engine

FAILURE (FOLDING BROKEN)

If query folding breaks before the filter step, Power BI Service cannot push the logic to the source database. The refresh will fail or dangerously attempt to pull the entire historical dataset into memory.

The deployment pipeline requires precise configuration across three phases



Policies are governed by two distinct time horizons

ARCHIVE WINDOW

Definition:	Controls the total policy history limit.
Example Setup:	Store 5 Years.
Engine Action:	Automatically purges and drops data partitions older than this configured limit.

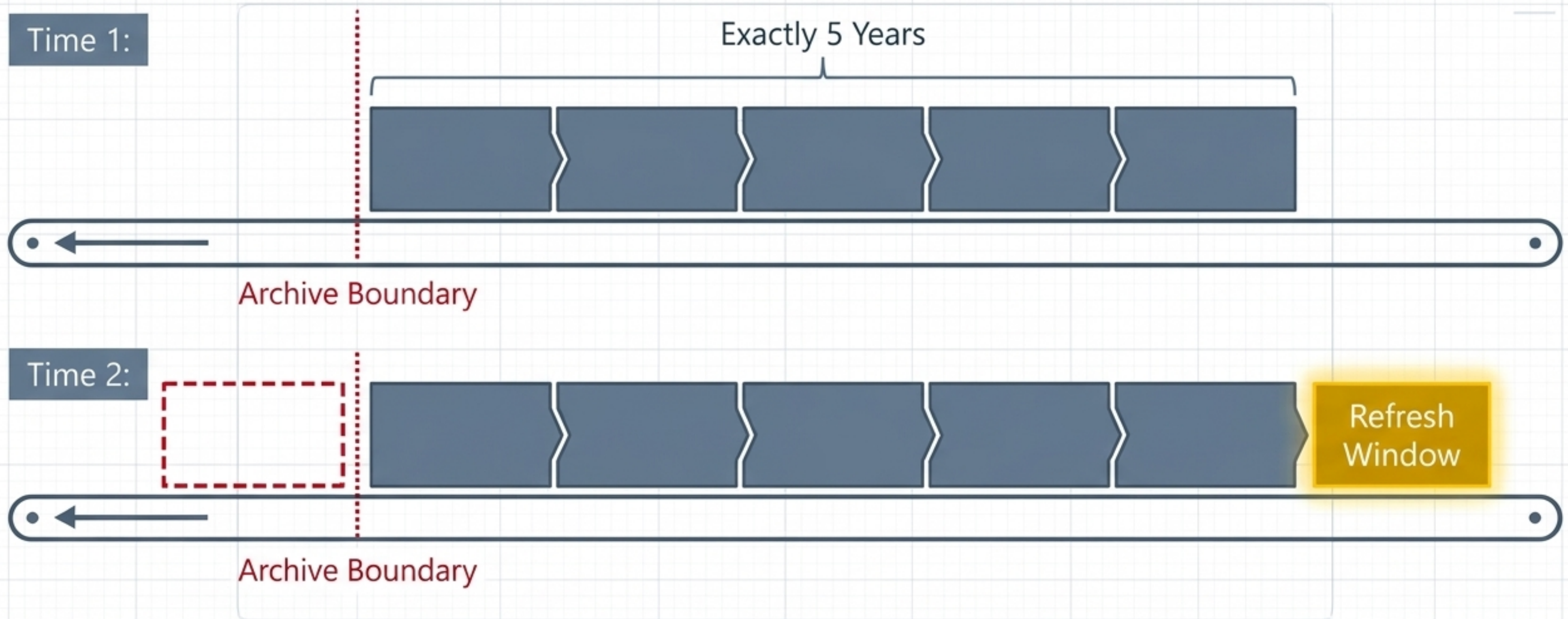


REFRESH WINDOW

Definition:	Defines the active, rolling update interval.
Example Setup:	Refresh 10 Days.
Engine Action:	Query runs strictly against data within this narrow, moving window. Leaves older data locked and untouched.



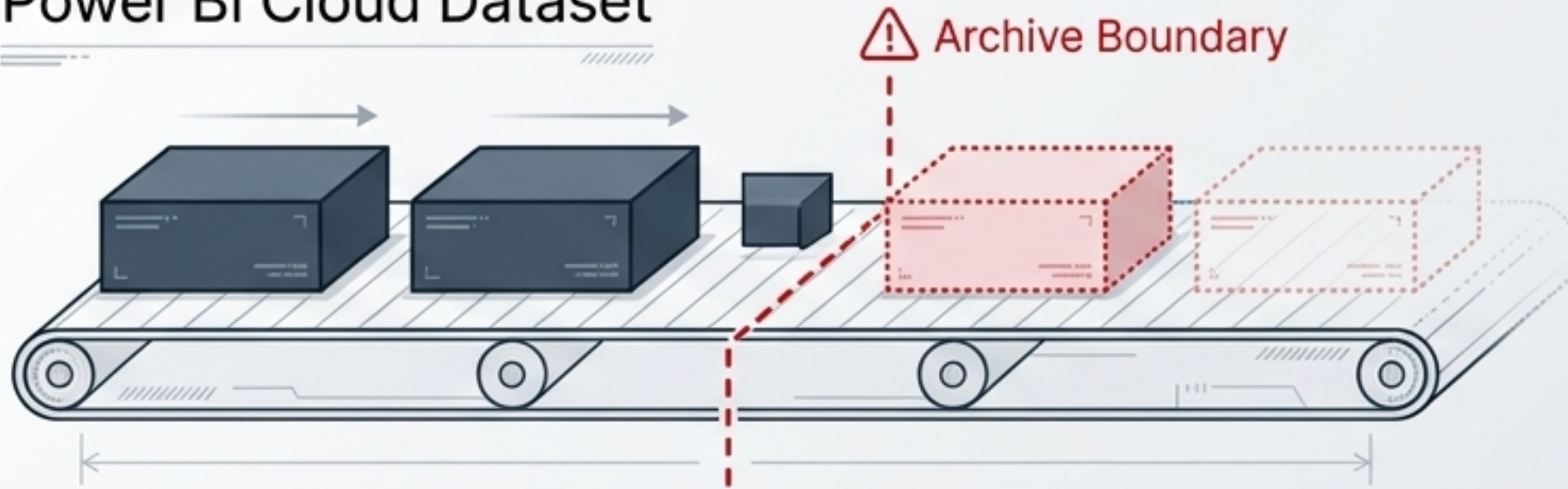
The rolling progression automatically purges expired partitions



As time moves forward into new calendar periods, Power BI evaluates partition age. Partitions falling completely outside the archive window are permanently dropped from the dataset.

Purging partitions in Power BI never alters the source database

Power BI Cloud Dataset



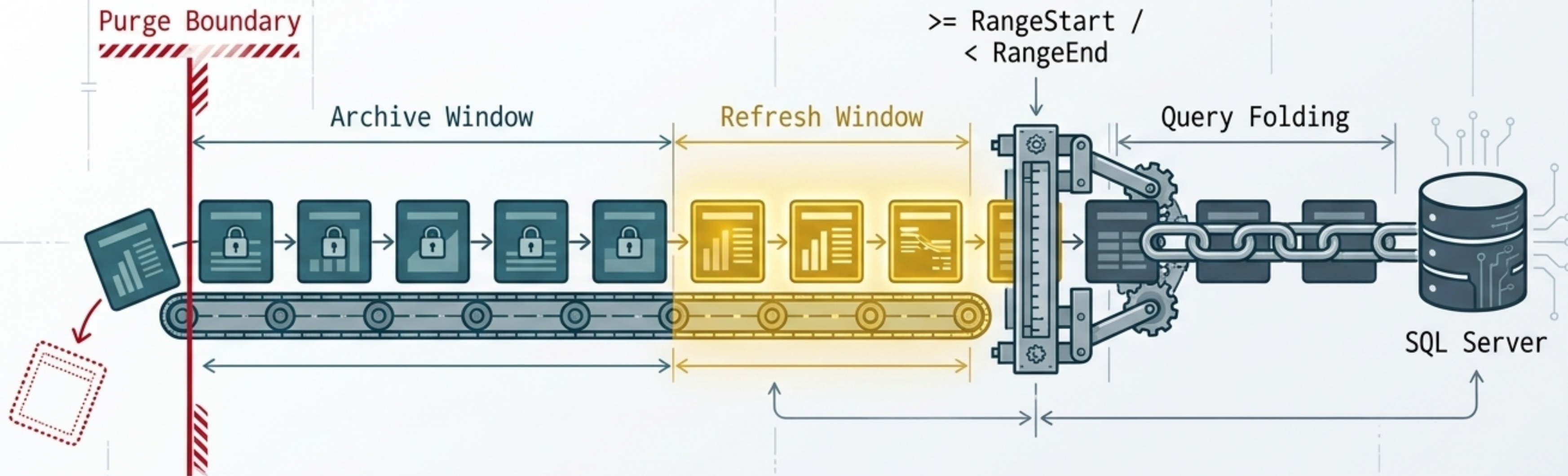
Historical data prior to the threshold is permanently deleted from the semantic model and is no longer accessible in report visuals or measures.

Source System Database



Source System Safety Guarantee: Deleting partitions inside Power BI has absolutely zero impact on the underlying source database (e.g., SQL Server remains completely untouched).

The Partition Engine in Motion



By enforcing strict parameters, preserving query folding, and establishing rigid time windows, Power BI transforms a massive static dataset into a lightweight, self-cleaning automated pipeline.

PL-300 Exam Essential Checklist

Incremental Refresh Exam Rules

- ❗ **Parameters:** Must be exactly RangeStart and RangeEnd (Case-Sensitive).
- ❗ **Data Type:** Must be strictly Date/Time (Convert if Date/Integer).
- ❗ **Operator Logic:** Always $\geq$ RangeStart AND $<$ RangeEnd (Prevents duplication).
- ❗ **Query Folding:** Mandatory up to the parameter filter step (Prevents full-history pull).
- Desktop vs Service:** Desktop uses parameters as static filters; Service overrides them to build dynamic partitions.
- Data Deletion:** Dropping partitions in PBI removes them from visuals but leaves the SQL source completely untouched.